RaceDay RESTful Web API (Part 2)

TEST EXECUTION REFERENCE & VERIFICATION CHECKLIST

ASP.NET Core • EF Core
Swagger & CLI Verification Matrix

Environment Target: Open `http://localhost:5000` or `http://localhost:5195/index.html` in browser — API ACTIVE & READY

1 TEST CREDENTIALS & ROLE SCOPES

Role	Full Name	Email (Username)	Password	Target Scope
Organiser	Thabo Mokoena	`thabo@runsa.co.za`	`Password123!`	Owner of Event 1 (Soweto Marathon) & Event 3
Organiser	Sarah Jenkins	`sarah@capecycling.co.za`	`Password123!`	Owner of Event 2 (Cape Town Cycle Tour)
Participant	Nomsa Dlamini	`nomsa.dlamini@gmail.com`	`Password123!`	Athlete account for category entry & withdrawal

2 SWAGGER UI TESTING SEQUENCE (STEPS 1 – 4)

Test 1: Public Event Browsing & Serialization Check — GET

Location: `/api/events` | **Role Required:** None (Public)
Inputs: None (Leave all query parameters blank) → Click *Execute*
Expected Status: **200 OK**
Verification: Returns array containing seeded events and categories without serialization cycles or circular reference errors.

Test 2: Athlete Registration & Conflict Checking — POST

Location: `/api/auth/register` | **Role Required:** None (Public)

```
{
  "fullName": "Sipho Khumalo",
  "email": "sipho.khumalo@test.co.za",
  "password": "Password123!",
  "role": "Participant",
  "phone": "+27 82 444 1122",
  "dateOfBirth": "1998-07-15T00:00:00.000Z"
}
```

Execution 1: Click *Execute* → **201 Created** with new generated `userId` and PBKDF2 hash.
Execution 2: Click *Execute* again (exact same data) → **409 Conflict** (`"An account with this email already exists."`).

Test 3: Organiser Authentication — POST

Location: `/api/auth/login` | **Role Required:** None (Public)

```
{
  "email": "thabo@runsa.co.za",
  "password": "Password123!"
}
```

Expected Status: **200 OK**
Verification: Response header sets the `.AspNetCore.Session` cookie with `Role: "Organiser"` and `UserId: 1`.

Test 4: Organiser Event Creation — POST

Location: `/api/events` | **Role Required:** Organiser (Logged in as Thabo)

```
{
  "eventName": "Durban City 10k Run",
  "eventDate": "2027-05-10T06:00:00.000Z",
  "startTime": "06:00:00",
  "venueId": 1,
  "description": "Fast coastal route."
}
```

Expected Status: 201 Created. Returns `eventId: 4` bound with foreign key `organiserId: 1`.

2 SWAGGER UI TESTING SEQUENCE (STEPS 5 – 10)

Test 5: Organiser Category Creation POST

Location: `/api/events/{id}/categories` | **Parameter:** `id: 1` | **Role:** Organiser (Thabo)

```
{
  "categoryName": "5km Junior Fun Run",
  "distanceKm": 5.0,
  "entryFee": 100.00,
  "maxParticipants": 1000
}
```

Expected Status: 201 Created. Verification: Returns `categoryID: 8` under Event 1.

Test 6: Security Boundary Guard (Role Mismatch Rejection) POST

Location: `/api/events/{eventId}/enrol` | **Parameter:** `eventId: 2` | **Active Role:** Organiser (Thabo)

Request Body: `{"categoryId": 4}`

Expected Status: 403 Forbidden. Body: `{"message": "Forbidden. This action requires the 'Participant' role."}`.

Test 7: Participant Authentication POST

7A. Clear Active Session: POST `/api/auth/logout` → Click *Execute* (**200 OK**).

7B. Login as Athlete: POST `/api/auth/login`

```
{
  "email": "nomsa.dlamini@gmail.com",
  "password": "Password123!"
}
```

Expected Status: 200 OK. Establishes session with `Role: "Participant"`.

Test 8: Participant Lifecycle (Enrol, Duplicate Guard, Withdraw, Re-enrol) POST

8A. Enrolment & Bib Generation: POST `/api/events/2/enrol` with `{"categoryId": 4}` → **201 Created** (`enrolmentID: 6` , `bibNumber: "BIB-2-4-XXXX"`).

8B. Duplicate Prevention Check: Click *Execute* again with identical payload → **409 Conflict** (`"You are already enrolled in this category."`).

8C. Self-Withdrawal: DELETE `/api/enrolments/6` → **200 OK** (`{"message": "Enrolment withdrawn."}`).

8D. Re-enrolment: POST `/api/events/2/enrol` with `{"categoryId": 4}` → **201 Created** with a new unique bib number.

Test 9: Official Race Result Capture POST

9A. Switch Session: Logout, then login via `/api/auth/login` as Sarah Jenkins (`sarah@capecycling.co.za` / `Password123!`).

9B. Capture Result: POST `/api/results` (Role Required: Organiser / Event 2 Owner)

```
{
  "enrolmentId": 3,
  "finishTime": "03:15:20",
  "position": 45,
  "status": "Completed"
}
```

Expected Status: 201 Created (Returns flat `ResultResponseDto` with validated event ownership).

Test 10: Public Leaderboard Lookup GET

Location: `/api/events/{eventId}/results` | **Parameter:** `eventId: 1` | **Role:** None (Public)

Expected Status: 200 OK. Returns ranked finishers sorted chronologically by finish position.

3 AUTOMATED TERMINAL UNIT TESTING SUITE

Terminal Execution: Run at repository root directory (`RaceDay/`) in PowerShell or Bash:

```
dotnet test RaceDay.sln
```

Expected Output: `Passed! - Failed: 0, Passed: 6, Skipped: 0, Total: 6`

Unit Test Method	Assertion Focus	Status
`Test1_PublicGetEvents_ReturnsAllEventsWithDtoProjections`	Unauthenticated read access and DTO serialization	PASSED
`Test2_RegisterDuplicateEmail_Returns409Conflict`	Unique email constraint enforcement	PASSED
`Test3_OrganiserCreatesEvent_BindsOwnershipAndReturns201`	Organiser role check and foreign key binding	PASSED
`Test4_ParticipantEnrolment_AllocatesBibAndReturns201`	Dynamic race bib formatting (`BIB-{event}-{cat}-XXXX`)	PASSED
`Test5_DuplicateEnrolment_Returns409Conflict`	Prevents duplicate category registration	PASSED
`Test6_CategoryCapacityExceeded_Returns400BadRequest`	Category capacity ceiling enforcement	PASSED

4 CI/CD PIPELINE WORKFLOW (.GITHUB ACTIONS)

Workflow Configuration

Active • Green

File: `.github/workflows/build-and-test.yml`
Trigger: `git push` or PR merge to `main` branch

1. `actions/checkout@v4` (Repository checkout)
2. `actions/setup-dotnet@v4` (Provision .NET 8 SDK)
3. `dotnet restore RaceDay.sln`
4. `dotnet build RaceDay.sln --no-restore -c Release`
5. `dotnet test RaceDay.sln --no-build -c Release`

Quality & Execution Gates

- **Build Enforcement:** Zero compilation warnings.
- **Test Runner:** Ubuntu latest environment.
- **Artifact Isolation:** Release configuration prevents test artifact leaking into deployment.
- **Verification Status:** All stages pass automatically.

✓ **Continuous Integration: PASSED**

5 RUBRIC & ARCHITECTURE COMPLIANCE

Git Commit History:
Over 20+ atomic, meaningful commits adhering to Conventional Commits (`feat(auth)`, `feat(events)`, `test(enrolment)`, `ci(workflow)`).

Layered Solution Architecture:
Clean separation of concerns with controllers delegating to EF Core DbContext, distinct DTO contracts, and isolated unit test projects.